

AI-платформа финансового комплаенса и безопасности транзакций

Полное описание проекта и оценка уровня Enterprise

Позиционирование в портфолио: FinTech • Финансовая безопасность • Управление ИИ • Комплаенс • Аудитируемость • Blockchain Integrity • Платёжная инфраструктура

Основной принцип

Пусть ИИ рассуждает. Пусть детерминированное программное обеспечение применяет правила. Пусть криптография доказывает целостность. Пусть люди авторизуют риск.

Этот документ описывает концепцию проекта, архитектуру, модель безопасности, финансовую предметную область, границы применения ИИ, процесс комплаенса, одобрение на основе риска, систему аудита, роль блокчейна, абстракцию платёжных систем, стратегию тестирования, пробелы перед выходом в production и рекомендуемую дорожную карту.

1. Краткое резюме

Проект представляет собой платформу финансового контроля с приоритетом на безопасность, предназначенную для обработки бизнес-счетов и финансовых транзакций при чётком разделении ответственности между ИИ, комплаенсом, авторизацией, выполнением платежей и аудитируемостью.

Цель проекта — **не создать AI-чат-бота, способного переводить деньги.**

Цель — создать контролируемый финансовый workflow, в котором ИИ может предоставлять рассуждения и рекомендации, тогда как детерминированное программное обеспечение и явные политики авторизации определяют, что системе разрешено выполнять.

Архитектура объединяет:

- Spring Boot;
- Spring Security;
- React/Redux;
- структурированный импорт счетов;
- детерминированную XML-валидацию;
- локальный ИИ через Ollama;
- гибридный поиск;
- правила комплаенса;
- оценку риска;
- human-in-the-loop approval;
- концепции токенизации;
- криптографические доказательства целостности SHA-256;

- привязку данных к блокчейну;
- подключаемые платёжные системы, например Stellar.

Проект наиболее убедительно позиционировать как **AI Financial Compliance & Transaction Security Platform**, а не как демонстрацию блокчейна или парсер счетов.

2. Постановка проблемы

Финансовые системы должны одновременно обеспечивать автоматизацию, безопасность, регуляторный контроль, аудитируемость, конфиденциальность и управление операционными рисками.

ИИ способен улучшить понимание документов и анализ комплаенса, однако неограниченный AI-компонент не должен иметь прямых полномочий на выполнение финансовых транзакций.

Проект решает эту проблему посредством разделения четырёх задач:

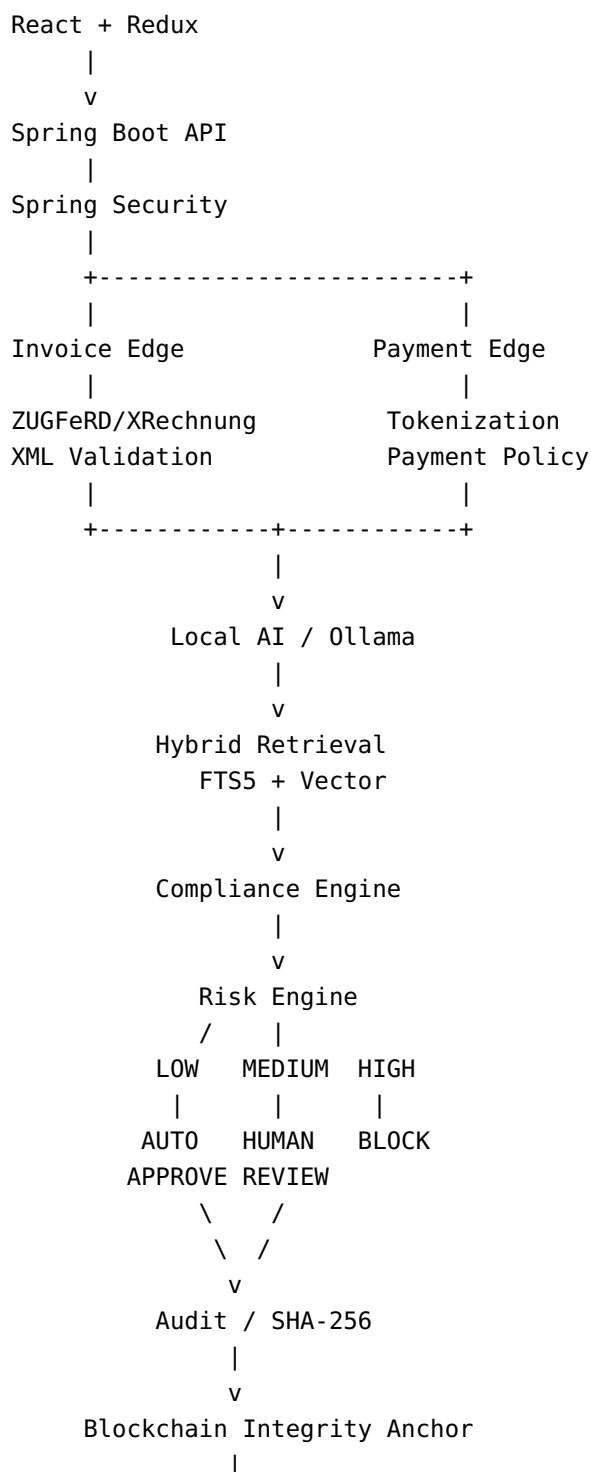
- **Рассуждение (Reasoning):** локальный ИИ интерпретирует структурированный и очищенный бизнес-контекст.
- **Применение правил (Enforcement):** детерминированная валидация и правила комплаенса определяют, что разрешено.
- **Авторизация (Authorization):** политики риска определяют, требуется ли автоматическое выполнение, проверка человеком или блокировка.
- **Доказательство (Proof):** криптографические записи аудита и опциональная привязка к блокчейну создают доказательства, позволяющие обнаружить изменения.

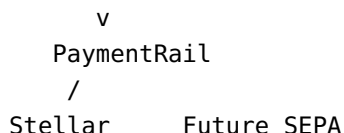
3. Ценность проекта

Направление	Ценность
FinTech	Финансовые суммы, транзакции, обработка счетов, платёжные политики и абстракция платёжных систем.
Security	Spring Security, границы безопасности, концепции токенизации, защита логов, fail-closed платёжная политика.
AI	Локальное рассуждение LLM без предоставления модели прямой возможности переводить деньги.
Compliance	Детерминированная валидация в сочетании с AI-анализом и явными решениями на основе риска.
Audit	Доказательства целостности SHA-256, append-only записи и привязка к блокчейну.
Enterprise	Разделение ответственности, человеческий контроль, настраиваемые правила и будущие платёжные адаптеры.

Направление	Ценность
Portfolio	Демонстрация Java/Spring, безопасности, финансового моделирования, управления ИИ и мышления в терминах распределённых систем.

4. Архитектура высокого уровня





5. Сквозной бизнес-процесс

1. Бизнес-счёт поступает через защищённый API.
2. Периметр импорта проверяет формат документа и отклоняет небезопасные XML-конструкции.
3. Структурированные данные, связанные с ZUGFeRD/XRechnung, преобразуются в контролируемые поля предметной области.
4. Детерминированная валидация проверяет обязательные поля и структурные/бизнес-ограничения **до начала работы ИИ**.
5. Чувствительные или ненужные данные очищаются перед передачей в локальный AI-контур.
6. Локальная LLM формирует структурированную оценку комплаенса, а не прямую инструкцию на платёж.
7. Гибридный поиск предоставляет релевантные правила и доказательства комплаенса.
8. Compliance Engine объединяет детерминированную валидацию и результаты AI-анализа.
9. Risk Engine присваивает уровень риска: **LOW, MEDIUM или HIGH**.
10. Низкорисковые случаи могут быть автоматически одобрены.
11. Среднерисковые случаи требуют одобрения человека.
12. Высокорисковые случаи блокируются.
13. Одобрённые решения создают audit-запись и криптографическое доказательство целостности.
14. Хеш может быть привязан к блокчейну или другому внешнему механизму обеспечения целостности без публикации самого счёта.
15. Только авторизованный платёжный workflow может обращаться к настроенному `PaymentRail`.

6. Финансовая модель предметной области

Финансовое ядро должно оставаться независимым от AI-инфраструктуры.

Текущая предметная область включает value object `MonetaryAmount` и абстракцию `FinancialTransaction`.

`MonetaryAmount` основан на:

- `BigDecimal`;
- явной информации о валюте;
- масштабе (`scale`);
- определённом режиме округления.

`FinancialTransaction` содержит:

- идентификатор транзакции;
- сумму;
- временную метку;
- периметр валидации входящих данных.

```
MonetaryAmount
- BigDecimal amount
- Currency code
- scale
- rounding policy

FinancialTransaction
- transactionId
- amount
- timestamp
- validateIngestionPerimeter()

|
+---- FiatPaymentTransaction
+---- future StellarPaymentTransaction
+---- future SEPA / other transaction types
```

Это разделение важно, поскольку финансовая корректность не должна зависеть от поведения LLM.

7. Импорт счетов и детерминированная валидация

Периметр обработки счетов намеренно является детерминированным.

XML нельзя считать доверенным входом только потому, что AI-модель умеет его читать.

Требования безопасности включают защиту от XML External Entities и небезопасного поведения парсера.

Парсер должен использовать безопасные настройки DOM/XML-парсера и отклонять повреждённые или опасные документы.

Для production-реализации слой обработки счетов должен поддерживать профильную валидацию ZUGFeRD/XRechnung, включая:

- валидацию схемы;
- бизнес-правила;
- code lists;
- согласованность VAT/налогов;

- ограничения конкретного профиля документа.

Извлечение встроенного XML из PDF/A-3 может быть реализовано как дополнительный адаптер.

Ключевое архитектурное правило: валидация схемы и детерминированного контракта выполняется до AI-рассуждения.

8. Локальный ИИ и безопасность ИИ

Локальный AI-контур построен вокруг Ollama или аналогичной модели, работающей локально.

Цель — уменьшить ненужную передачу коммерческих данных счетов наружу и сохранить выполнение модели под контролем инфраструктуры и механизмов безопасности организации.

ИИ является советником, а не авторитетом.

Контролируемый pipeline:

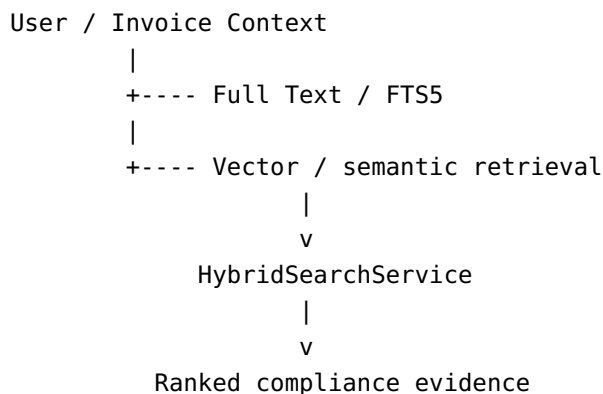
```
Invoice
  ↓
sanitize
  ↓
deterministic validation
  ↓
structured context
  ↓
local LLM
  ↓
typed ComplianceAiAssessment
  ↓
Compliance Engine
  ↓
Risk Engine
  ↓
Authorization
```

AI-слой также должен обеспечивать:

- обнаружение prompt injection и фильтрацию подозрительных инструкций;
 - строгую валидацию структурированного результата;
 - обработку тайм-аутов и ошибок модели;
 - запись версии модели и версии prompt;
 - пороги confidence;
 - отсутствие прямого доступа к платёжным credentials;
 - отсутствие неограниченного доступа во внешнюю сеть;
 - авторизацию на уровне инструментов, если инструменты будут добавлены позднее.
-

9. Гибридный поиск

Рассуждение о комплаенсе выигрывает от retrieval-механизма, поскольку правила, внутренние политики и доказательства должны передаваться модели как контролируемый контекст, а не зависеть только от памяти модели.



Текущая реализация представляет собой облегчённый retrieval-механизм, а не полноценную production vector database.

В production-версии его можно заменить или дополнить надёжным vector store, сохранив:

- детерминированные фильтры;
- версионирование документов;
- даты вступления правил в силу.

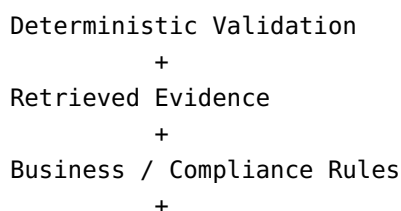
10. Compliance Engine

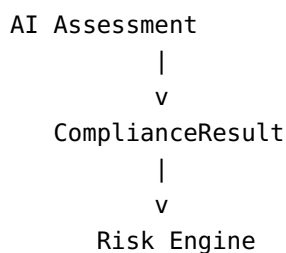
Compliance Engine является одним из центральных элементов архитектуры.

Комплаенс не должен сводиться к одному ответу LLM.

Вместо этого необходимо объединять:

- детерминированную валидацию;
- найденные доказательства;
- настроенные правила;
- интерпретацию ИИ.





В будущем правила должны иметь версии и быть связаны с датами вступления в силу и авторитетными источниками.

Это делает решения воспроизводимыми и аудируемыми.

11. Risk Engine и Human-in-the-Loop

Автоматизация на основе риска является важной Enterprise-функцией.

Не каждая транзакция должна обрабатываться одинаково.

Риск	Действие по умолчанию	Причина
LOW	Автоматическое одобрение	Детерминированные проверки пройдены, риск находится в заданных пределах.
MEDIUM	Проверка человеком	Требуется явная авторизация или дополнительные доказательства.
HIGH	Блокировка	Возможное мошенничество, нарушение политики, несовпадение хеша, опасный вход или другое критическое условие.

Дополнительные правила могут принудительно отправлять транзакцию на проверку человеком при:

- высокой сумме;
- необычном контрагенте;
- налоговых несоответствиях;
- отсутствии доказательств;
- криптографических несоответствиях.

12. Безопасность платежей и токенизация

Платёжные данные должны быть изолированы от остальной предметной области.

Граница токенизации может преобразовать чувствительные данные карты в application token до того, как транзакция попадёт в основную систему.

```

Raw payment credential
  |
  v
Tokenization Edge
  |
  v
Token ID
  /   |
Core  Logs  Compliance
financial monitoring workflows
services

```

Критический regression-тест должен проверять, что исходный PAN никогда не появляется в application logs.

Текущий компонент токенизации является демонстрационным и **не является PCI-сертифицированным vault или HSM-решением**.

В production секреты и криптографические материалы платежей должны защищаться соответствующей инфраструктурой HSM/KMS/Vault и архитектурой PCI/security организации.

13. Безопасность авторизации платежей

Самый важный security invariant:

ошибка комплаенса не должна незаметно превращаться в платёж.

```

Compliance = DENY
  |
  +----> PaymentRail MUST NOT be called

Compliance = REVIEW
  |
  +----> humanApproved == true
        |
        +----> PaymentRail

Compliance = APPROVE
  |
  +----> risk/policy checks
        |
        +----> PaymentRail

```

Это гораздо надёжнее, чем полагаться на AI prompt вроде:

«Не оплачивай рискованные счета».

Сам сервисный слой должен технически обеспечивать это правило.

14. Аудитируемость и криптографическая целостность

Система аудита должна фиксировать:

- что произошло;
- кто или что инициировало действие;
- какие версии правил и модели использовались;
- какое решение было принято.

```
AuditEvent  
eventId  
actor  
timestamp  
action  
resourceId  
decision  
riskLevel  
ruleVersion  
modelVersion  
previousHash  
currentHash
```

Hash chain обеспечивает защиту порядка событий от незаметного изменения:

```
Event 1 -> Hash 1  
Event 2 + Hash 1 -> Hash 2  
Event 3 + Hash 2 -> Hash 3
```

Внешний blockchain затем может закрепить выбранный audit hash.

Сам счёт, персональные данные, PAN, IBAN или коммерческие секреты **не должны записываться в публичный блокчейн.**

Привязка к блокчейну является механизмом обеспечения целостности.

Сама по себе она не должна преподноситься как доказательство регуляторного комплаенса или юридической корректности хранения записей.

15. Роль Blockchain и Web3

Web3j здесь наиболее полезен как адаптер для слоя обеспечения целостности, а не как причина помещать все финансовые данные on-chain.

Stellar может оставаться адаптером платежей/settlement, тогда как Ethereum-compatible сеть может опционально предоставлять внешний audit anchor.

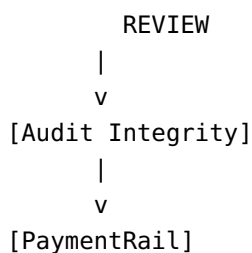
```
PaymentRail
|
+---- StellarPaymentAdapter
+---- future SEPA adapter
+---- other rails

AuditAnchor
|
+---- local cryptographic ledger
+---- Web3j / EVM anchor
+---- future enterprise ledger
```

Такое разделение не позволяет проекту превратиться в «**blockchain ради blockchain**».

16. Архитектура безопасности

```
External Input
|
v
[Spring Security]
|
v
[Input / XML Security]
|
v
[Sanitization + DLP boundary]
|
v
[Local AI Guardrails]
|
v
[Deterministic Compliance]
|
v
[Risk / Authorization]
/   |
BLOCK HUMAN APPROVE
```



Таким образом, безопасность является многоуровневой.

AI-компонент сам по себе **не является границей безопасности** — авторитетным остаётся финансовый сервисный слой.

17. API Design

Проект может предоставлять отдельные API для:

- счетов;
- одобрений;
- платежей;
- audit-данных.

Примеры:

Endpoint	Назначение
POST /api/invoices	Загрузка и обработка данных счёта.
POST /api/approvals	Регистрация явного одобрения человеком.
POST /api/payments	Выполнение авторизованного платёжного запроса.
GET /api/audit/...	Получение аудиторских доказательств и статуса целостности.

Production API должны дополнительно включать:

- OAuth2/OIDC;
- RBAC/ABAC;
- rate limiting;
- request IDs;
- idempotency keys;
- валидацию;
- структурированные ответы об ошибках;
- audit correlation IDs.

18. Observability

Enterprise-развёртывание требует большего, чем обычные application logs.

Рекомендуемая система observability включает:

- структурированные JSON-логи без чувствительных платёжных данных;
- correlation/request IDs;
- метрики обработки счетов, решений комплаенса, задержки AI и платёжных ошибок;
- traces через весь pipeline: ingestion → AI → compliance → authorization → payment;
- security alerts для повторяющихся ошибок или подозрительного ввода;
- health/readiness endpoints;
- метрики версий моделей и правил.

19. Стратегия тестирования

Тест	Что доказывает
FinancialPrecisionTest	Деньги используют BigDecimal и определённую семантику scale/rounding.
ArchitectureTest	Границы финансовой безопасности и правила зависимостей соблюдаются.
XmlSecurityTest	Небезопасные XML-конструкции отклоняются.
TokenizationRegressionTest	Чувствительные платёжные данные преобразуются в токен.
LogComplianceTest	Исходный PAN отсутствует в логах.
PromptInjectionTest	Подозрительные инструкции не проходят через AI boundary.
ComplianceRegressionTest	Некорректные, отсутствующие и корректные случаи дают ожидаемые решения.
RiskEngineTest	Классификация риска детерминирована настроенными политиками.
PaymentAuthorizationTest	DENY никогда не вызывает PaymentRail; REVIEW требует одобрения человека.
AuditIntegrityTest	Hash chain обнаруживает изменения.
Integration tests	Spring Security, persistence и сервисные границы работают совместно.

Наиболее сильными являются **invariant tests**: они доказывают, что опасные состояния не могут достигнуть границы выполнения платежа.

20. Enterprise-оценка

Область	Оценка	Комментарий
FinTech domain design	8.5/10	Сильная финансовая модель, концепции счетов и платежей.
Application security	8/10	Хорошее направление многоуровневой безопасности.
Payment security	7.5/10	Хорошая архитектура; vault/HSM ещё необходимо реализовать для production.
AI security	7.5/10	Локальный ИИ и контролируемая граница сильны, но нужны дополнительные механизмы.
Compliance architecture	8.5/10	Связка deterministic + AI + risk + human review выглядит убедительно.
Blockchain/Web3	8/10	Сильный вариант для integrity/audit layer, а не обычная blockchain-демонстрация.
Auditability	8.5/10	Hash chain и модель доказательств дают значительную ценность.
Architecture	9/10	Чёткое разделение security, compliance, risk, approval и payment.
Production readiness	5.5/10	Требуются persistence, IAM, HSM/KMS, observability и hardening deployment.
Testing	6.5/10	Хорошая база, но требуется более широкое integration/security coverage.
Enterprise relevance	9/10	Очень актуально для платежей, ERP, банков и compliance workflows.
Portfolio value	9/10	Демонстрирует потенциал senior-level cross-domain engineering.

21. Текущие ограничения

- Текущая реализация retrieval является облегчённой и не должна описываться как production vector database.
- Поддержка счетов является MVP-парсером/валидатором, а не полной реализацией всех ZUGFeRD/XRechnung-профилей и PDF/A-3 workflow.
- Компонент токенизации является демонстрационным и не представляет собой сертифицированный PCI vault.
- Stellar adapter работает преимущественно в режиме симуляции и не представляет production signing и transaction submission.
- Для production deployment хранилища audit и approval всё ещё требуют надёжного persistence.

- Контракт AI output должен быть усилен строгой schema validation и версионированием модели/prompt.
 - Необходимо выполнить полную компиляцию, разрешение зависимостей и containerized integration testing в CI-среде с поддержкой Maven/Docker.
-

22. Production Roadmap

Phase 1 — Secure MVP

- Invoice ingestion
- XML security
- BigDecimal domain
- Local Ollama
- Compliance + risk
- Payment policy
- Audit hash chain

Phase 2 — Enterprise Controls

- PostgreSQL
- OAuth2/OIDC
- RBAC/ABAC
- Vault/KMS
- durable approvals
- structured audit events
- OpenTelemetry

Phase 3 — Financial Integrations

- полные ZUGFeRD/XRechnung profiles
- SEPA adapter
- production Stellar integration
- idempotency
- reconciliation

Phase 4 — Integrity Infrastructure

- Web3j/EVM audit anchoring
- независимый verification service
- key rotation
- signed audit manifests

Phase 5 — AI Governance

- model registry
- prompt versioning
- evaluation datasets
- adversarial testing
- model risk controls
- explainability и human override

23. Рекомендуемая структура репозитория

```
backend/  
├─ security/  
├─ invoice/  
├─ ai/  
├─ retrieval/  
├─ compliance/  
├─ risk/  
├─ approval/  
├─ tokenization/  
├─ transaction/  
├─ payment/  
├─ audit/  
├─ ledger/  
├─ config/  
├─ controller/  
└─ model/  
  
frontend/  
├─ components/  
├─ features/  
├─ store/  
└─ services/  
  
docs/  
├─ ARCHITECTURE.md  
├─ SECURITY_ASSESSMENT.md  
├─ TEST_STRATEGY.md  
├─ THREAT_MODEL.md  
└─ API.md  
  
examples/  
└─ invoice.xml
```

24. Модель угроз

Угроза	Основной механизм защиты
Вредоносный XML	Безопасный парсер, schema validation, защита от XXE.
Prompt injection	Input sanitization, фильтрация инструкций, ограниченный AI context.
Галлюцинации AI	Детерминированные правила, структурированный output и human review.

Угроза	Основной механизм защиты
Несанкционированный платёж	Payment policy, authorization service и fail-closed behavior.
Утечка PAN	Tokenization boundary и log regression tests.
Изменение audit-данных	Hash chain и опциональный внешний blockchain anchor.
Компрометация секретов	Production KMS/HSM/Vault и key rotation.
Replay / повторный платёж	Idempotency keys и transaction state machine.
Утечка данных во внешний AI	Локальная модель, изоляция сети и контролируемый egress.

25. Ключевые Security Invariants

- ИИ никогда не должен иметь прямой авторизации на перевод денег.
- DENY никогда не должен вызывать PaymentRail.
- REVIEW должен требовать явной авторизации человека.
- Исходный PAN никогда не должен появляться в логах.
- Содержимое счёта никогда не должно записываться в публичный блокчейн.
- Детерминированная schema/security validation должна выполняться до AI reasoning.
- Платёжные секреты не должны храниться в исходном коде или обычной конфигурации приложения.
- Audit-записи должны быть защищены от изменений и воспроизводимы.

26. Definition of Done

Система считается выполненной, когда:

- вредоносный XML-документ безопасно отклоняется;
- корректный счёт преобразуется в типизированные финансовые данные;
- финансовые расчёты используют явную decimal semantics;
- локальный ИИ способен анализировать контролируемый контекст и возвращать валидированный структурированный результат;
- решения комплаенса воспроизводимы на основе правил, доказательств и записанных версий;
- Risk Engine определяет автоматическое одобрение, human review или блокировку;
- заблокированная транзакция не может достигнуть платёжного адаптера;
- транзакция, требующая review, не может попасть в payment workflow без явного одобрения;
- чувствительные платёжные credentials токенизируются до попадания в основной workflow;
- чувствительные credentials не появляются в логах;
- каждое финансовое решение создаёт audit record;
- audit records обладают проверяемой криптографической целостностью;

- при использовании blockchain anchoring в нём сохраняются только допустимые metadata целостности;
- система имеет автоматизированные unit, architecture, security и integration tests;
- production deployment использует надлежащие identity, secrets management, persistence и observability.

27. Позиционирование проекта в портфолио

Для технического интервью или портфолио проект следует представлять как **контролируемую финансовую платформу**, а не как набор технологий.

Самая важная инженерная идея — это границы между:

- AI;
- security;
- compliance;
- payment execution.

Ключевая модель:

```
AI = reasoning
Rules = enforcement
Risk = decision policy
Human = authorization for exceptions
Cryptography = integrity proof
Blockchain = external anchor
PaymentRail = controlled execution
```

Это делает проект релевантным для ролей, связанных с:

- Java/Spring;
- FinTech;
- платежами;
- application security;
- AI infrastructure;
- автоматизацией комплаенса;
- Enterprise Architecture.

28. Итоговая оценка

Архитектурная концепция: 9/10

Разделение reasoning, enforcement, authorization и proof является сильным и профессионально релевантным.

Текущая зрелость реализации: примерно 6–6.5/10

Архитектура опережает production-реализацию.

Основные пробелы:

- persistence;
- IAM;
- HSM/KMS;
- полноценная поддержка invoice profiles;
- observability;
- более сильное AI governance;
- production payment integrations.

FinTech-релевантность: 8.5/10

Проект содержит значимые концепции:

- финансового моделирования;
- обработки счетов;
- комплаенса;
- управления рисками;
- платежей.

Security-релевантность: 8/10

Многоуровневый подход к безопасности и payment authorization invariants являются сильным материалом для портфолио.

Enterprise-потенциал: 9/10

После production hardening архитектура естественным образом может применяться в:

- банковских системах;
- платёжных системах;
- ERP;
- системах финансового комплаенса.

Ценность для портфолио: 9/10

Проект демонстрирует cross-functional engineering, а не просто использование одного framework.

Финальная архитектурная формулировка

Пусть ИИ рассуждает. Пусть детерминированное программное обеспечение применяет правила. Пусть криптография доказывает целостность. Пусть люди авторизуют риск.

Именно этот принцип является наиболее сильной одной фразой для объяснения проекта senior-инженеру, security architect, FinTech-рекрутеру или hiring manager.